EVM 인덱서 시스템 분석

1. 인덱서 개요

EVM 인덱서는 블록체인 데이터를 실시간으로 인덱싱하여 JSON-RPC API의 빠른 쿼리를 지원하는 핵심 컴포넌트입니다.

1.1 주요 기능

- **실시간 인덱싱**: 블록 생성과 동시에 데이터 인덱싱
- **트랜잭션 추적**: EVM 트랜잭션 및 영수증 저장
- **이벤트 로그**: 스마트 컨트랙트 이벤트 인덱싱
- **블룸 필터**: 로그 검색 최적화
- **메모리풀 관리**: 대기 중인 트랜잭션 추적

2. EVMIndexer 인터페이스

2.1 핵심 인터페이스

```
type EVMIndexer interface {

snapshot.ExtensionSnapshotter

storetypes.ABCIListener



// 트랜잭션 관련

TxByHash(ctx context.Context, hash common.Hash) (*rpctypes.RPCTransaction, error
```

```go
IterateBlockTxs(ctx context.Context, blockHeight  uint64, cb  func(tx *rpctypes.
TxHashByBlockAndIndex(ctx context.Context, blockHeight  uint64, index  uint64) (

// 트랜잭션 영수증
TxReceiptByHash(ctx context.Context, hash common.Hash) (*coretypes.Receipt, erro
IterateBlockTxReceipts(ctx context.Context, blockHeight  uint64, cb  func(tx *co

// 블록 관련
BlockHashToNumber(ctx context.Context, hash common.Hash) (uint64, error)
BlockHeaderByNumber(ctx context.Context, number  uint64) (*coretypes.Header, err

// Cosmos ↔ EVM 트랜잭션 해시 매핑
CosmosTxHashByTxHash(ctx context.Context, hash common.Hash) ([]byte, error)
TxHashByCosmosTxHash(ctx context.Context, hash []byte) (common.Hash, error)

// 이벤트 구독
Subscribe() (chan *coretypes.Header, chan []*coretypes.Log, chan *rpctypes.RPCTr

// 인덱싱 상태
GetLastIndexedHeight(ctx context.Context) (uint64, error)

// 메모리풀 관리
TxInPending(hash common.Hash) *rpctypes.RPCTransaction
```

```
TxInQueued(hash common.Hash) *rpctypes.RPCTransaction

PushPendingTx(tx *coretypes.Transaction)

PushQueuedTx(tx *coretypes.Transaction)

PendingTxs() []*rpctypes.RPCTransaction

QueuedTxs() []*rpctypes.RPCTransaction

NumPendingTxs() int

NumQueuedTxs() int

RemovePendingTx(hash common.Hash)

RemoveQueuedTx(hash common.Hash)


// 블룸 필터

ReadBloomBits(ctx context.Context, section  uint64, index  uint32) ([]byte, erro

PeekBloomBitsNextSection(ctx context.Context) (uint64, error)

IsBloomIndexingRunning() bool


// 생명주기 관리

Close() error

Wait()

Initialize(clientCtx client.Context, contextCreator contextCreator, consensusPar

}
```

3. EVMIndexerImpl 구조체

3.1 주요 필드

```go
type EVMIndexerImpl struct {
enabled  bool
retainHeight  uint64
backfillStartHeight  uint64

// 상태 추적을 위한 atomic 변수들
pruningRunning *atomic.Bool
bloomIndexingRunning *atomic.Bool
lastIndexedHeight *atomic.Uint64
lastPrunedHeight *atomic.Uint64
lastBloomIndexedHeight *atomic.Uint64

db dbm.DB
logger log.Logger
txConfig client.TxConfig
ac address.Codec
appCodec codec.Codec

clientCtx client.Context
contextCreator contextCreator
consensusParamsGetter consensusParamsGetter
```

```
store *CacheStoreWithBatch

evmKeeper *evmkeeper.Keeper


schema collections.Schema


// 데이터 저장소들

BlockHeaderMap collections.Map[uint64, coretypes.Header]

BlockAndIndexToTxHashMap collections.Map[collections.Pair[uint64, uint64], []byt

BlockHashToNumberMap collections.Map[[]byte, uint64]


TxMap collections.Map[[]byte, rpctypes.RPCTransaction]

TxReceiptMap collections.Map[[]byte, coretypes.Receipt]

TxHashToCosmosTxHash collections.Map[[]byte, []byte]

CosmosTxHashToTxHash collections.Map[[]byte, []byte]


BloomBits collections.Map[collections.Pair[uint64, uint32], []byte]

BloomBitsNextSection collections.Sequence


// 이벤트 채널들

blockChans []chan *coretypes.Header

logsChans []chan []*coretypes.Log

pendingChans []chan *rpctypes.RPCTransaction
```

```
// 메모리풀 캐시 (TTL 캐시 사용)
pendingTxs *ttlcache.Cache[common.Hash, *rpctypes.RPCTransaction]
queuedTxs *ttlcache.Cache[common.Hash, *rpctypes.RPCTransaction]

// 인덱싱 작업 채널
indexingChan  chan *indexingTask
indexingWg *sync.WaitGroup
}
```

3.2 인덱싱 작업 구조

```
type indexingTask struct {
req *abci.RequestFinalizeBlock
res *abci.ResponseFinalizeBlock

// 상태 의존적 인수들
args *indexingArgs
}
```

4. 데이터 저장소 구조

4.1 블록 관련 저장소

```
// 블록 헤더 저장
BlockHeaderMap  collections.Map[uint64, coretypes.Header]


// 블록 번호와 트랜잭션 인덱스로 트랜잭션 해시 조회
BlockAndIndexToTxHashMap  collections.Map[collections.Pair[uint64, uint64], []by


// 블록 해시로 블록 번호 조회
BlockHashToNumberMap  collections.Map[[]byte, uint64]
```

4.2 트랜잭션 관련 저장소

```
// 트랜잭션 데이터 저장
TxMap  collections.Map[[]byte, rpctypes.RPCTransaction]


// 트랜잭션 영수증 저장
TxReceiptMap  collections.Map[[]byte, coretypes.Receipt]


// Cosmos ↔ EVM 트랜잭션 해시 매핑
TxHashToCosmosTxHash  collections.Map[[]byte, []byte]
CosmosTxHashToTxHash  collections.Map[[]byte, []byte]
```

4.3 블룸 필터 저장소

```
// 블룸 비트 저장
BloomBits  collections.Map[collections.Pair[uint64, uint32], []byte]


// 다음 블룸 섹션 번호
BloomBitsNextSection  collections.Sequence
```

5. 인덱서 초기화 과정

5.1 NewEVMIndexer 함수

```
func  NewEVMIndexer(
db dbm.DB,
ac address.Codec,
appCodec codec.Codec,
logger log.Logger,
txConfig client.TxConfig,
evmKeeper *evmkeeper.Keeper,
) (EVMIndexer, error)
```

5.2 초기화 단계

1. **설정 로드**: EVM Keeper에서 인덱서 설정 가져오기
2. **저장소 생성**: CacheStoreWithBatch 생성

3. **컬렉션 스키마 구성**: Collections 스키마 빌드

4. **TTL 캐시 초기화**: 메모리풀 트랜잭션용 캐시

5. **채널 생성**: 인덱싱 작업용 버퍼드 채널

6. **백그라운드 프로세스 시작**: TTL 캐시 만료 처리

6. 실시간 인덱싱 프로세스

6.1 ABCI 리스너 구현

인덱서는 `storetypes.ABCIListener` 를 구현하여 블록 생성 이벤트를 수신합니다.

6.2 인덱싱 루프

```
func (e *EVMIndexerImpl) indexingLoop() {
for task := range e.indexingChan {
e.indexingWg.Add(1)
go func(task *indexingTask) {
defer e.indexingWg.Done()
e.processIndexingTask(task)
}(task)
}
}
```

6.3 인덱싱 작업 처리

1. **블록 데이터 추출**: FinalizeBlock 요청/응답에서 데이터 추출

2. **트랜잭션 파싱**: EVM 트랜잭션 식별 및 파싱

3. **영수증 생성**: 트랜잭션 실행 결과 영수증 생성

4. **이벤트 로그 추출**: 스마트 컨트랙트 이벤트 로그 추출

5. **데이터 저장**: 인덱싱된 데이터를 저장소에 저장

6. **이벤트 발생**: 구독자들에게 새 데이터 알림

7. 메모리풀 관리

7.1 TTL 캐시 사용

```
// pending tx lifetime is 1 minutes in indexer
pendingTxs: ttlcache.New(
ttlcache.WithTTL[common.Hash, *rpctypes.RPCTransaction](time.Minute),
),

// queued tx lifetime is 1 minutes in indexer
queuedTxs: ttlcache.New(
ttlcache.WithTTL[common.Hash, *rpctypes.RPCTransaction](time.Minute),
),
```

7.2 메모리풀 상태 관리

- **Pending**: 실행 준비가 된 트랜잭션들
- **Queued**: 논스 순서 대기 중인 트랜잭션들
- **자동 만료**: 1분 후 자동으로 캐시에서 제거

8. 이벤트 구독 시스템

8.1 구독 메커니즘

```
func (e *EVMIndexerImpl) Subscribe() (chan *coretypes.Header, chan []*coretypes.

blockChan := make(chan *coretypes.Header)

logsChan := make(chan []*coretypes.Log)

pendingChan := make(chan *rpctypes.RPCTransaction)



e.blockChans = append(e.blockChans, blockChan)

e.logsChans = append(e.logsChans, logsChan)

e.pendingChans = append(e.pendingChans, pendingChan)

return  blockChan, logsChan, pendingChan

}
```

8.2 이벤트 발생

```
func (e *EVMIndexerImpl) blockEventsEmitter(blockEvents *blockEvents, done  chan

defer  close(done)



if  blockEvents == nil {

return

}
```

```
// emit logs first
for _, logs := range blockEvents.logs {
for _, logsChan := range e.logsChans {
logsChan <- logs
}
}

// emit block header
for _, blockChan := range e.blockChans {
blockChan <- blockEvents.header
}
}
```

9. 블룸 필터 시스템

9.1 블룸 필터 목적

- 로그 검색 성능 최적화
- 대량의 로그에서 빠른 필터링
- Ethereum 호환성 유지

9.2 블룸 인덱싱 상태 추적

```go
func (e *EVMIndexerImpl) IsBloomIndexingRunning() bool {
return e.bloomIndexingRunning.Load()
}
```

10. 백필(Backfill) 기능

10.1 백필 프로세스

```go
if e.backfillStartHeight != 0 {
lastIndexedHeight, err := e.GetLastIndexedHeight(context.Background())
if err != nil {
e.logger.Error("failed to get last indexed height", "err", err)
return err
}

if e.backfillStartHeight < lastIndexedHeight {
err = e.Backfill(e.backfillStartHeight, lastIndexedHeight)
if err != nil {
e.logger.Error("failed to backfill", "err", err)
return err
}
}
```

```
}
```

10.2 백필 용도

- 인덱서 재시작 시 누락된 데이터 복구
- 새로운 인덱싱 규칙 적용
- 데이터 무결성 보장

11. 인덱서 종료 및 정리

11.1 Graceful Shutdown

```
func (e *EVMIndexerImpl) Close() error {
if  e.pendingTxs != nil {
e.pendingTxs.Stop()
}
if  e.queuedTxs != nil {
e.queuedTxs.Stop()
}

return  e.flushStore()
}
```

11.2 Store Flush 과정

```go
func (e *EVMIndexerImpl) flushStore() error {
e.logger.Info("service stop", "msg", "Stopping EVM indexer service")

// wait for all the indexing to finish
e.Wait()

// wait for pruning to complete before flushing the store
ticker := time.NewTicker(time.Millisecond * 100)
timeout := time.NewTimer(time.Second * 30)
defer ticker.Stop()
defer timeout.Stop()

for {
select {
case <-ticker.C:
if !e.pruningRunning.Swap(true) {
e.store.Write()
e.logger.Info("Closing evm_index.db")
if err := e.store.db.Close(); err != nil {
return err
}
return nil
```

```
}
case <-timeout.C:
return  fmt.Errorf("timeout waiting for EVM indexer to complete before shutdown"
}
}
}
```

이러한 인덱서 시스템을 통해 MiniEVM은 Ethereum JSON-RPC API의 모든 쿼리를 빠르고 효율적으로 처리할 수 있습니다.